good picture of how much work has been done so far and how much remains. In addition, by tracking the number of completed automated acceptance tests against the number of pending tests, you can get an idea of the progress you're making over time.

When you write tests in this narrative style, another benefit emerges. Each automated acceptance test becomes a documented, worked example of how the system can be used to solve a particular business requirement. And when the tests are web tests, the worked examples will even be illustrated with screenshots taken along the way.

But what about the testers? Automated acceptance testing and QA.

Automatically deploying your application into production when the automated acceptance tests pass requires a great deal of discipline and the utmost confidence in the quality and comprehensiveness of your automated tests. This is a worthy goal, and a number of organizations do manage this, but for most, things are not quite that simple.

In typical enterprise environments, the testers will probably still want to do at least some exploratory testing before releasing the application to production. But if the automated test results are clear and visible, they can save the QA team days or weeks of time that would normally be spent on regression or basic mechanical testing, and let them focus on more interesting testing activities. This, in turn, can speed up the release cycle significantly.

2.5 Maintenance

In many organizations, the developers who worked on the initial project don't maintain the application once it goes into production. Instead, the task is handed over to a maintenance or BAU (Business as Usual) team. In this sort of environment, executable specifications and living documentation are a great way to streamline the handover process, as they provide a set of worked examples of the application's features and illustrations of the code that supports these features.

Executable specifications also make it much easier for maintenance teams to implement changes or bug fixes. Let's see how this works with a simple example. Suppose that users have requested to be informed about trains that are due to arrive within the next 30 minutes, and not just the next 15, as is currently the case.

The scenario related to this requirement is as follows:

```
Find out what time the next trains for my destination station leave

Narrative:
In order to plan my trips more effectively
As a commuter
I want to know the next trains going to my destination

Scenario: Find the optimal itinerary between stations on the same line
Given Western line trains from Emu Plains leave Parramatta for Town Hall at
7:58, 8:00, 8:02, 8:11, 8:14, 8:21
When I want to travel from Parramatta to Town Hall at 8:00
Then I should be told about the trains at: 8:02, 8:11, 8:14
```